

Self-Healing Data Pipeline

Deployment & Testing Guide — Self-Service Setup for Your Snowflake Account

For: Shai Leib, CA DHCS ETS Data Architect

Author: Christian Abdel-Nour, Sr. Technical Architect — Snowflake Solutions Delivery

July 2026 • Repo: github.com/sfc-gh-cabdelnour/selfhealing-pipeline

What This Is

A Snowflake-native self-healing data pipeline that detects schema drift, halts ingestion, diagnoses root cause via Cortex AI, regenerates downstream SQL, validates against a clone, and opens a GitHub PR for human approval. The entire system runs inside Snowflake — no external dependencies.

Contents

1. Prerequisites
2. Deployment Steps (01–08)
3. Testing the Deployment
4. Adapting for Your Environment
5. Object Inventory
6. Design Decision Summary
7. Troubleshooting

1. Prerequisites

- **Snowflake account** with Enterprise edition or higher (Cortex AI requires it)
- **Snowflake CLI** (`snow`) installed and configured
- **ACCOUNTADMIN** role (or equivalent with CREATE DATABASE, CREATE ROLE, CREATE INTEGRATION)
- **Cortex AI enabled** — verify: `SELECT SNOWFLAKE.CORTEX.COMPLETE('llama3.1-70b','test');`
- **[Optional] GitHub PAT** for PR automation (pipeline works without it)
- **Git** to clone the repository

Design Rationale: Why ACCOUNTADMIN?

Initial setup creates databases, roles, and External Access Integrations — operations requiring elevated privileges. After setup, the pipeline runs under SELFHEALING_ADMIN (custom role with least-privilege grants). You only need ACCOUNTADMIN for the one-time deployment.

2. Deployment Steps

Step 1: Clone the repository

```
git clone https://github.com/sfc-gh-cabdelnour/selfhealing-pipeline.git
cd selfhealing-pipeline
```

Step 2: Set your connection

```
export SNOWFLAKE_CONNECTION=your-connection-name
snow sql -c $SNOWFLAKE_CONNECTION -q "SELECT CURRENT_ACCOUNT(), CURRENT_ROLE();" 
```

Step 3: Deploy foundation

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/01_foundation.sql
```

Creates: roles (SELFHEALING_ADMIN, SELFHEALING_EXECUTOR), database, 4 schemas, warehouse, 7 tables with constraints and retention policies.

Design Rationale: CREATE IF NOT EXISTS Everywhere

The script is idempotent. Running it twice does nothing harmful. If someone accidentally re-runs deployment, no data is destroyed. Compare to CREATE OR REPLACE which would drop everything.

Step 4: Deploy detection

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/02_detection.sql
```

Design Rationale: Detection via INFORMATION_SCHEMA Diff

Compares live schema against stored baseline. Pure SQL — no external catalog, no Horizon integration required.
Catches: new columns, dropped columns, data type changes. Idempotent: running 10 times with no changes produces zero events.

Step 5: Deploy diagnosis + halt gate

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/03_diagnosis.sql
```

Design Rationale: TRY_PARSE_JSON Fallback

If the LLM returns malformed JSON (~5% of calls), the function returns a safe fallback object instead of crashing. Production-grade error handling that keeps the pipeline running even when AI misbehaves.

Step 6: Deploy lineage + code generation

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/04_lineage_codegen.sql
```

Design Rationale: Python for Code Generation

Snowflake SQL Scripting has limitations with cursor variable binding inside FOR loops. Python with Snowpark gives reliable iteration, proper error handling, and cleaner Cortex integration. Tradeoff: ~2s cold-start overhead (acceptable for a 15-min scheduled task).

Step 7: Deploy GitHub integration (optional)

```
# First create the secret (run in Snowsight):  
#   CREATE OR REPLACE SECRET SELFHEALING_PROD.CONFIG.GITHUB_PAT_SECRET  
#     TYPE = GENERIC_STRING SECRET_STRING = 'ghp_yourPAThere';  
  
snow sql -c $SNOWFLAKE_CONNECTION -f config/05_github_eai.sql
```

Without GitHub Integration

The pipeline still detects, diagnoses, generates code, and validates. You just won't get automated PRs. Review generated code directly in the GENERATED_CODE table.

Step 8: Deploy validation

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/06_validation.sql
```

Step 9: Deploy orchestrator + task

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/07_pipeline_tasks.sql
```

Design Rationale: Task Starts Suspended

The scheduled task starts in SUSPENDED state. You must explicitly ALTER TASK ... RESUME to activate. This prevents unintended runs during deployment or testing.

Step 10: Seed demo data

```
snow sql -c $SNOWFLAKE_CONNECTION -f config/08_seed_demo.sql
```

3. Testing the Deployment

Test: End-to-End Drift Simulation

```
USE SCHEMA SELFHEALING_PROD.CONFIG;

-- Simulate drift
ALTER TABLE SELFHEALING_PROD.BRONZE.ORDERS ADD COLUMN LOYALTY_POINTS NUMBER;

-- Detect
CALL SP_SCHEMA_DRIFT_DETECTOR();
-- Expected: NEW=1

-- Halt gate
CALL SP_HALT_GATE();
-- Expected: HALTED

-- Get event + generate code (30-60s for Cortex)
SET eid = (SELECT EVENT_ID FROM SCHEMA_CHANGE_EVENTS WHERE PIPELINE_STATUS='PENDING' LIMIT 1);
CALL SP_GENERATE_ARTIFACT_CODE($eid);
-- Expected: Generated 1 artifacts

-- Validate
CALL SP_VALIDATE_GENERATED_CODE($eid);
-- Expected: PASS=0 FAIL=1 (GOLD aggregate can't auto-resolve)

-- Verify
SELECT ARTIFACT_NAME, TEST_STATUS FROM GENERATED_CODE WHERE EVENT_ID = $eid;
```

Test: Idempotency

```
-- Run detector again (should find 0 new events)
CALL SP_SCHEMA_DRIFT_DETECTOR();
-- Expected: NEW=0

-- Run codegen again (should skip existing)
CALL SP_GENERATE_ARTIFACT_CODE($eid);
-- Expected: Generated 0 artifacts
```

4. Adapting for Your Environment

Point at your existing dbt project:

1. Populate ARTIFACT_REGISTRY with your model graph (generate from `dbt ls --output json`)
2. Seed GENERATED_CODE with current SQL for each model (gives LLM context for regeneration)
3. Point SCHEMA_REGISTRY at your BRONZE schema: `INSERT INTO SCHEMA_REGISTRY SELECT ... FROM YOUR_DB.INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA = 'BRONZE'`

Add the halt gate to existing pipelines:

```
-- Add to the TOP of any ingestion procedure:
DECLARE v_gate VARCHAR;
BEGIN
    CALL SELFHEALING_PROD.CONFIG.SP_HALT_GATE() INTO v_gate;
    IF (LEFT(v_gate, 6) = 'HALTED') THEN
        RETURN 'Pipeline blocked: ' || v_gate;
    END IF;
    -- ... rest of your ingestion logic ...
END;
```

Design Rationale: Framework-Agnostic

The pipeline doesn't require dbt. It operates on plain SQL + a metadata table (ARTIFACT_REGISTRY). If you use dbt, populate from the manifest. If you use stored procedures, populate manually. If you use Dynamic Tables, use SHOW DYNAMIC TABLES. The pattern is decoupled from any specific framework.

5. Object Inventory

Schema	Object	Type	Purpose
CONFIG	SCHEMA_REGISTRY	Table	Known-good baseline
CONFIG	SCHEMA_CHANGE_EVENTS	Table	Detected drift with lifecycle
CONFIG	PIPELINE_HEALTH_AUDIT	Table	Quality findings + AI diagnosis
CONFIG	PIPELINE_RUN_LOG	Table	Execution observability
CONFIG	ARTIFACT_REGISTRY	Table	Model lineage graph
CONFIG	GENERATED_CODE	Table	LLM output + test results
CONFIG	VW_ARTIFACT_LINEAGE	View	Recursive downstream traversal
CONFIG	SP_SCHEMA_DRIFT_DETECTOR	Proc	Diff INFORMATION_SCHEMA vs registry
CONFIG	SP_HALT_GATE	Proc	Block on unresolved issues
CONFIG	FN_DIAGNOSE_FAILURE	Function	Cortex AI root cause
CONFIG	SP_GENERATE_ARTIFACT_CODE	Proc (Python)	LLM code regeneration
CONFIG	SP_VALIDATE_GENERATED_CODE	Proc (Python)	Clone + test

6. Design Decision Summary

Decision	Choice	Rationale
Detection language	SQL	Set operations on INFORMATION_SCHEMA, fastest cold-start
Codegen language	Python	Reliable iteration, error handling, Cortex integration
Diagnosis LLM	claude-4-sonnet	Best reasoning for structured JSON output
Codegen LLM	llama3.1-70b	Good SQL, lower cost, fast for mechanical rewrites
Lineage	Recursive view	SQL-native, no manifest, always current
Validation	Zero-copy clone	Real data, zero cost, instant, auto-cleaned
Human gate	PR-first	Auditable, reviewable, reversible
Idempotency	Skip-if-exists	Safe re-execution, no duplicates
Error handling	TRY_PARSE_JSON + try/except	Never crashes on LLM issues

7. Troubleshooting

Symptom	Cause	Fix
"No downstream for TABLE"	Missing ARTIFACT_REGISTRY entry	INSERT the lineage row for that table
Diagnosis returns confidence: 0.0	Cortex malformed response (fallback triggered)	Check Cortex availability, retry
"Clone failed"	Insufficient privileges	Run as ACCOUNTADMIN or grant CREATE DATABASE
Detector finds 0 after ALTER TABLE	Event already exists (idempotency)	Check for existing PENDING entry, resolve first
GitHub PR 401	PAT expired	Regenerate with <code>repo</code> scope, update secret

Full Reset (Teardown + Rebuild)

```
DROP DATABASE IF EXISTS SELFHEALING_PROD;  
DROP DATABASE IF EXISTS SELFHEALING_DEV;  
DROP WAREHOUSE IF EXISTS SELFHEALING_WH;  
DROP ROLE IF EXISTS SELFHEALING_EXECUTOR;  
DROP ROLE IF EXISTS SELFHEALING_ADMIN;
```

```
-- Then re-run Steps 3-10
```